

Quantum recurrent neural networks for sequential learning

Yanan Li¹, Zhimin Wang^{*,1}, Rongbing Han, Shangshang Shi, Jiaxin Li, Ruimin Shang, Haiyong Zheng, Guoqiang Zhong, Yongjian Gu^{*}

Faculty of Information Science and Engineering, Ocean University of China, Qingdao, 266100, China

ARTICLE INFO

Article history:

Received 13 November 2022
Received in revised form 14 May 2023
Accepted 4 July 2023
Available online 16 July 2023

Keywords:

Quantum deep neural networks
Quantum recurrent neural networks
Temporal sequential data
Meteorological indicators
Stock price
Text categorization

ABSTRACT

Quantum neural network (QNN) is one of the promising directions where the near-term noisy intermediate-scale quantum (NISQ) devices could find advantageous applications against classical resources. Recurrent neural networks are the most fundamental networks for sequential learning, but up to now there is still a lack of canonical model of quantum recurrent neural network (QRNN), which certainly restricts the research in the field of quantum deep learning. In the present work, we propose a new kind of QRNN which would be a good candidate as the canonical QRNN model, where, the quantum recurrent blocks (QRBs) are constructed in the hardware-efficient way, and the QRNN is built by stacking the QRBs in a staggered way that can greatly reduce the algorithm's requirement with regard to the coherent time of quantum devices. That is, our QRNN is much more accessible on NISQ devices. Furthermore, the performance of the present QRNN model is verified concretely using three different kinds of classical sequential data, i.e., meteorological indicators, stock price, and text categorization. The numerical experiments show that our QRNN achieves much better performance in prediction (classification) accuracy against the classical RNN and state-of-the-art QNN models for sequential learning, and can predict the changing details of temporal sequence data. The practical circuit structure and superior performance indicate that the present QRNN is a promising learning model to find quantum advantageous applications in the near term.

© 2023 Elsevier Ltd. All rights reserved.

1. Introduction

In recent years, deep neural networks (DNNs) (LeCun, Bengio, & Hinton, 2015) have enabled revolutionary applications in several domains of artificial intelligence (Jordan & Mitchell, 2015), such as computer vision (Voulodimos, Doulamis, Doulamis, & Protopapadakis, 2018) and natural language processing (Goldberg, 2016), etc. Parallely, remarkable breakthroughs have been seen in quantum computing (Arute et al., 2019; Bharti et al., 2022; Madsen et al., 2022; Zhong et al., 2020). With the demonstrations of quantum supremacy, we are entering the NISQ era of quantum computing, where NISQ refers to the noisy intermediate-scale quantum devices (Preskill, 2018). There is a growing consensus that NISQ devices may find useful applications in the near term. One of the most promising directions is the quantum neural network (QNN) (Farhi & Neven, 2018; Mitarai, Negoro, Kitagawa, & Fujii, 2018; Schuld, Bocharov, Svore, & Wiebe, 2020). QNN takes the parameterized quantum circuit (PQC) (Benedetti, Lloyd, Sack,

& Fiorentini, 2019) as a learning model, which is a quantum analogue of the classical neural network.

The great success of classical DNNs is mainly attributed to its flexible architecture. That is, the multilayer architecture is versatile to discover intricate structures in high-dimensional data. Specifically, the convolutional neural networks (CNNs) (Gu et al., 2018) can effectively capture spatial correlation within, say, image data; while the recurrent neural networks (RNNs) perform well when learning sequential data (Graves, 2014), e.g., natural language processing.

Inspired by DNNs, naturally quantum deep neural networks (QDNNs) should have similar architectures to process corresponding types of data. Indeed, for the quantum convolutional neural networks (QCNNs), there have been a fair amount of research covering QCNNs' structure, learnability, and applications (Cong, Choi, & Lukin, 2019; Herrmann et al., 2022; Hur, Kim, & Park, 2022; Pesah et al., 2021). In contrast, the studies about quantum recurrent neural networks (QRNNs) are rather sparse. Bausch developed a high-degree nonlinear quantum neuron (Bausch, 2020) based on the work of Cao, Guerreschi, and Aspuru-Guzik (2017), and used this neuron to build the recurrent networks. Such models possess good non-linearity but need to implement amplitude amplification operations, resulting in high complexity of quantum circuits. Takaki, Mitarai, Negoro, Fujii,

* Corresponding authors.

E-mail addresses: wangzhimin@ouc.edu.cn (Z. Wang), yjgu@ouc.edu.cn (Y. Gu).

¹ These authors contribute equally to this work.

and Kitagawa (2021) proposed a kind of QRNNs by employing a PQC with a recurrent structure. Such models use simple quantum circuits that are easy to implement on the NISQ devices, but the performance on the non-trivial sequential data has yet to be verified. In addition, Chen, Yoo, and Fang (2022) and Di Sipio, Huang, Chen, Mangini, and Worring (2022) developed hybrid quantum–classical models of QRNNs, where the classical linear layers in RNNs are replaced with PQCs. Such hybrid models just take the quantum circuits as acceleration sub-modules plugging into the classical networks, and face with the dilemma of the interface between quantum and classical systems.

Until now there is still a lack of canonical model of QRNN, which certainly restricts the research of QRNNs. Inspired by the QCNN model proposed by Cong et al. (2019), we consider that one canonical QRNN model should possess the following features: (1) flexible to be implemented on various NISQ platforms; (2) fully quantum evolved but not the quantum–classical hybrid networks to ease the interface problem; (3) efficient for sequential learning of classical data.

In order to address the above issues, in the present work, we develop a new kind of QRNN that can fulfill the requirements of the canonical QRNN. Specifically,

1. We first propose to construct the quantum recurrent block in a more hardware-efficient way, inspired by the Hamiltonian dynamics-based models developed by Takaki et al. (2021). More importantly, we propose a staggered architecture of QRNN by stacking the recurrent blocks in a staggered way. The staggered QRNN can greatly reduce the algorithm's requirement with regard to the coherent time of quantum devices. This property is of great significance, because increasing the coherent time of quantum hardware is extremely hard from the technology development point of view.
2. The present QRNN is a fully quantum learning model, where the outcome of quantum transformation is taken as the prediction of the data with minor post-processing. Our QRNN is a standard NISQ algorithm (Bharti et al., 2022), which can take full advantage of the near term quantum computers.
3. The performance of the present QRNN on classical sequential data is verified concretely. Three different kinds of sequential data including meteorological indicators, stock price, and natural language are applied to test the models. Our QRNN model shows better performance in prediction (classification) accuracy against the classical RNN and state-of-the-art QNN models for sequential learning, and can predict the changing details of the sequence data.

The NISQ-compliant structure as well as the good performance imply that our QRNN would be a promising candidate to find useful applications in the near term.

The rest of the paper is organized as follows. In Section 2, we describe the structure of the quantum recurrent block, which is the basic cell to construct the QRNNs. Section 3 shows the details of the QRNNs, including the architectures of QRNN and the method of optimizing parameters. In Section 4, we present the performance of the QRNNs on three kinds of classical sequential data, i.e., the data of meteorological indicators, stock price, and natural language. Finally, conclusions and outlook of the present work are discussed in Section 5.

2. Quantum recurrent block

In general, RNNs possess a multilayer architecture and each layer is the basic recurrent block. Depending on the specific design of the recurrent block, there are a number of RNN variants,

such as long short-term memory (LSTM) (Hochreiter & Schmidhuber, 1997) and gated recurrent unit (GRU) (Cho et al., 2014). The main idea of the recurrent block is that the prediction at a given moment is determined by both the new input data of the current moment and the information about the history of all the past elements of the sequence.

In the most basic recurrent block, the output at the time step t can be expressed as

$$\begin{aligned}\tilde{y}^{(t)} &= f_y(U_o \tilde{h}^{(t)}), \\ \tilde{h}^{(t)} &= f_h(V_{in} \tilde{x}^{(t)} + W \tilde{h}^{(t-1)}).\end{aligned}\quad (1)$$

In this equation, $\tilde{x}^{(t)}$ and $\tilde{y}^{(t)}$ represent the input and output of time step t , respectively, with the dimensions $x \in \mathbb{R}^n$ and $y^{(t)} \in \mathbb{R}^m$; $\tilde{h}^{(t-1)}$ is the hidden output of the previous step, which implicitly contains information about all the previous elements of the sequence; V_{in} , W and U_o are parameter matrices with the dimensions $V_{in}, W, U_o \in \mathbb{R}^{m \times n}$; f_y and f_h represent the activation functions. The training process of RNNs is to optimize the parameter matrices by minimizing a loss function. The basic classical recurrent block and one RNN by stacking the recurrent blocks are schematically shown in Fig. 1.

In the spirit of the classical recurrent block structure described above, the quantum recurrent block (QRB) can be designed in a way as shown schematically in Fig. 2. Generally, the qubits of the QRB are divided into two groups, i.e., two quantum registers denoting as Reg. D and Reg. H. Reg. D is used to embed the sequential data, one element at each time step, and Reg. H is used to store information about the history of all previous elements. This structure is inspired by the one proposed by Takaki et al. (2021), which is easy to implement on NISQ devices. In Takaki et al. (2021), the QRB is constructed based on Hamiltonian dynamics, and here we advance the model in a more hardware-efficient way. In general, the QRB consists of three parts: data encoding $U_{in}(x^{(t)})$, ansatz circuit, and partial quantum measurement. Below, we go into the details of implementing these three parts.

2.1. Data encoding

Data encoding is a process of loading classical data into quantum computer, which is to represent the classical data as quantum states. It is worth noting that in quantum machine learning, data encoding plays a crucial role, far beyond the thing of preparing the input. On the one hand, encoding the classical data quantumly is by no means a trivial thing, which would turn to be the bottleneck for the runtime of the whole algorithm. On the other hand, data encoding can be interpreted as a feature map, dubbed as “quantum feature map”, which is to map the input to Hilbert space of the quantum system (Schuld, 2021). A well-chosen quantum feature map can make the data linearly separable in the feature space, and thereby efficiently solve the learning problem.

Data encoding is equivalent to performing a unitary transformation $U_{in}(x)$ on the initial state, i.e., $|f(x)\rangle = U_{in}|0\rangle^{\otimes n}$ with n being the number of qubits. There exist numerous structures of circuit to embed the classical data into a quantum state. Among them, the most famous one would be the amplitude encoding, which can embed exponentially many classical data (Schuld, 2021). Specifically, given a normalized classical vector $x = (x_1, \dots, x_N)^T$ of dimension $N = 2^n$, amplitude encoding can represent this vector as amplitudes of an n -qubit quantum state, i.e., $U_{in}(x)|0\rangle^{\otimes n} = \sum_{i=0}^{2^n-1} x_i |i\rangle$. Similarly, a data matrix $A \in \mathbb{C}^{2^n \times 2^n}$ with entries a_{ij} satisfying $\sum_{ij} |a_{ij}|^2 = 1$ can be encoded as $U_{in}(A)|0\rangle^{\otimes m}|0\rangle^{\otimes n} = \sum_{i=0}^{2^m-1} \sum_{j=0}^{2^n-1} a_{ij} |i\rangle |j\rangle$, where $|i\rangle$ and $|j\rangle$ are respectively the i th and j th computational basis state. However, amplitude encoding is much less common in QNNs, because the

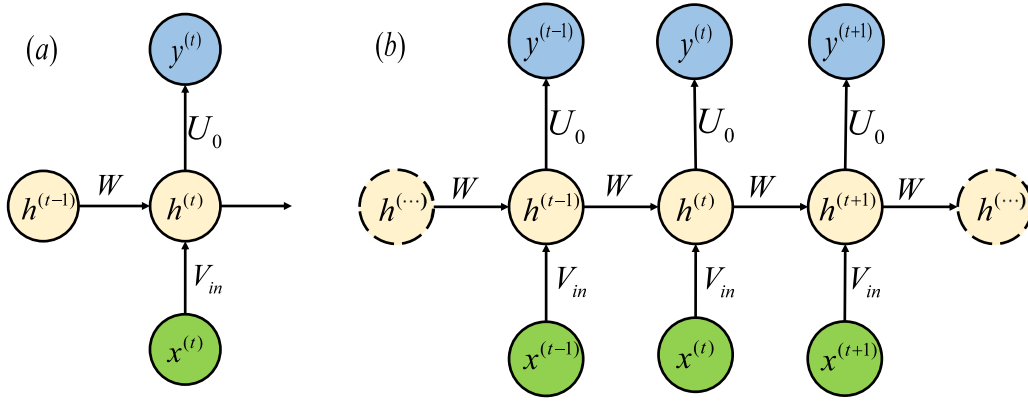


Fig. 1. (a) Structure of the basic classical recurrent block. (b) One basic architecture of RNN by stacking the basic recurrent blocks.

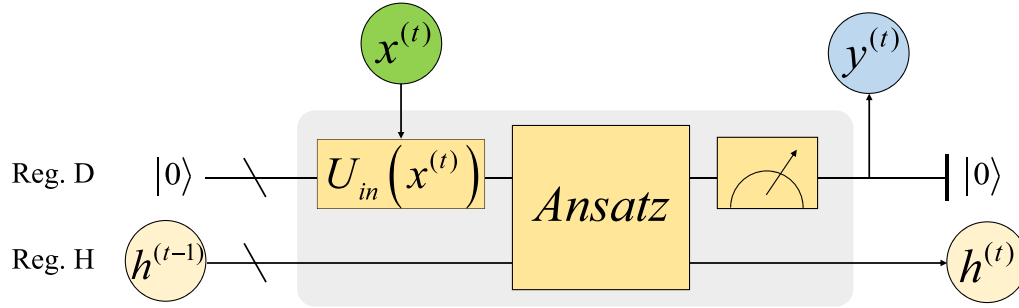


Fig. 2. Structure of the quantum recurrent block inspired by the basic classical recurrent block shown in Fig. 1.

quantum circuit cost for amplitude encoding usually grows as $O(poly(N))$ that is exponential of the number of qubits. Therefore, although having exponentially large data-encoding space, amplitude encoding cannot be implemented efficiently on NISQ devices.

In QNNs, the most commonly used encoding techniques are the angle encoding (Schuld, 2021) and the associated circuit encoding (Havlíček et al., 2019). In general, angle encoding and circuit encoding embed the classical data as the rotation angles of the single-qubit or controlled two-qubit rotation gates. Specifically, given one classical data point x_i , the angle encoding first rescales the data to $\tilde{x}_i$ lying between 0 and π , and then embeds it into a single qubit as, say $U_{in}(\tilde{x}_i)|0\rangle = \cos(\frac{\tilde{x}_i}{2})|0\rangle + \sin(\frac{\tilde{x}_i}{2})|1\rangle$ with $U_{in}(\tilde{x}_i)$ being the R_y gate $R_y(\theta) = [\cos \frac{\theta}{2}, -\sin \frac{\theta}{2}, \sin \frac{\theta}{2}, \cos \frac{\theta}{2}]$ (Nielsen & Chuang, 2010). For N data points $x = (x_1, \dots, x_N)^T$, the angle encoding embeds them by N qubits,

$$R_y^{\otimes N}(\tilde{x})|0\rangle^{\otimes N} = \bigotimes_{i=1}^N (\cos(\frac{\tilde{x}_i}{2})|0\rangle + \sin(\frac{\tilde{x}_i}{2})|1\rangle). \quad (2)$$

Formally, angle encoding is actually a kind of time-evolution encoding. Time-evolution encoding prescribes to associate a scalar value $x \in \mathbb{R}$ with the time t in the unitary evolution by a Hamiltonian $\hat{H}$, i.e., $U(x) = \exp(-i\hat{H}t)$. In the angle encoding, $\hat{H}$ is just the Pauli operation, and the corresponding unitaries $U(\theta) = \exp(-i\hat{\sigma}\theta)$ can be implemented efficiently on NISQ devices. Therefore, the computational cost of angle encoding is minor, while the required number of qubit is $O(N)$ for N data points.

Circuit encoding takes the angle encoding as the basis, and embeds the data into a more complex circuit than that of angle encoding. For example, more than one data point, say x_1, x_2, x_3 , can be encoded into one qubit as follows,

$$R_z(\tilde{x}_3)R_x(\tilde{x}_2)R_z(\tilde{x}_1)H|0\rangle. \quad (3)$$

Such strategy of dense encoding can address, to some extent, the problem of angle encoding which needs N qubits to embed N data points. Based on this strategy, up to 67-dimensional data has been handled successfully using quantum kernel method on the current quantum processors (Peters et al., 2021). More importantly, circuit encoding can be used to construct a complex feature map which is hard to simulate by the classical computers. For example, in Ref. Havlíček et al. (2019), one encoder is proposed as follows,

$$U_{\phi(x)} = \exp(i \sum_{j,k} \phi_{j,k}(x) Z_j Z_k) H^{\otimes n}, \quad (4)$$

where n is the number of qubit, $Z_j(Z_k)$ is the Pauli-Z operator for the j th (k th) qubit, $\phi_{j,k}$ are real functions, and H is the Hadamard gate. Even two layers of such an encoder circuit would make it computationally hard for the classical resources (Havlíček et al., 2019). However, it is worth mentioning that whether such a complex feature map leads to an advantage for discovering structure in data is still an open question.

In the present work, we employ angle encoding to load the sequential data. This decision is primarily driven by our goal to verify the efficiency of the QRNNs we developed in learning classical sequential data. As such, we opt for the data encoding as simple as possible and especially the commonly used one on NISQ devices. The circuit for angle encoding used in this work is illustrated in Fig. 3. It is important to note that each element of the sequential data is embedded in a replicative manner. It has been shown that input redundancy can provide an advantage in classification accuracy (Gil Vidal & Theis, 2020; Peruzzo et al., 2014).

In general, more sophisticated data encoding techniques, such as the circuit encoding, should be helpful to enhance the performance of QRNNs. We leave this exploration for future work.

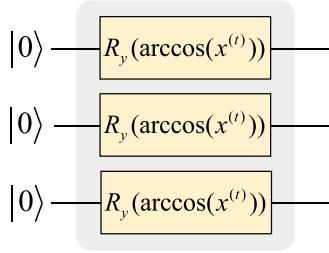


Fig. 3. The circuit for encoding the sequential data at the time step t .

2.2. Ansatz

The ansatz in Fig. 2 is to process the feature produced by the encoder circuit from the raw data, and output the new feature for the following task of classification or regression. Ansatz is, in fact, a parameterized quantum circuit, which has adjustable quantum gates. The adjustable parameters are optimized based on the data to learn, as they are determined in the algorithm of neural networks.

Ansatz, namely the PQC, is used to approximate the target function, which can map the feature data into different value domains representing different classifications. Similar to the universal approximation theorem in classical neural networks (Hornik, Stinchcombe, & White, 1989), there always exists a quantum circuit that can approximate the target function within an arbitrary small error (Cai, Ye, & Deng, 2022; Schuld, Sweke, & Meyer, 2021). Moreover, it is possible to construct such a circuit with polynomial cost of quantum gates (Benedetti et al., 2019; Peruzzo et al., 2014). Therefore, the ansatz aims to use polynomial number of quantum gates (i.e., polynomial number of parameters) to implement a function that can approximate the task at hand.

In practice, ansatz follows a fixed structure of quantum gates. There are two strategies to design the circuits, which are problem-inspired and hardware-efficient methods. Problem-inspired method is to leverage the Hamiltonian of the problem to construct the circuit, and optimizing the parameters of the circuit always means to search the ground state of the Hamiltonian. The ansatz applied in the variational quantum algorithms (VQAs) for solving eigenvalues and eigenstates (Cerezo, Arrasmith et al., 2021; Peruzzo et al., 2014), and for approximation and optimization (Farhi, Goldstone, & Gutmann, 2014) usually adopts the problem-inspired method. Whereas, the ansatz used in QNNs is typically hardware-efficient circuit, which consists of layers of native entangling gates (i.e., two-qubit gates) and single-qubit gates (Kandala et al., 2017). Essentially, such ansatz can be customized concerning the gate set and connectivity of the specific quantum devices, and the ansatz can be implemented directly on the device without the need of compilation.

Here, the ansatz circuit used in Fig. 2 is constructed in the hardware-efficient way. Specifically, the circuit is composed of single-qubit rotation gates and two-qubit gates (i.e., controlled rotation gates). As it is well-known, any single-qubit gate can be expressed as a combination of rotation gates about the $\hat{x}$, $\hat{y}$ and $\hat{z}$ axes (Nielsen & Chuang, 2010). We adopt the $X-Z$ decomposition to represent the single-qubit gates in the circuit,

$$U_{1q} = R_x(\alpha)R_z(\beta)R_x(\gamma), \quad (5)$$

where α , β , and γ are the adjustable parameters to be optimized in the learning process.

The two-qubit gates are applied to create entanglement between qubits in the circuit. The two-qubit gates would be fixed without adjustable parameters, such as the CNOT and controlled Pauli-Z gate; they would be also adjustable, such as the controlled

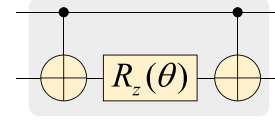


Fig. 4. Implementation of the two-qubit gate $R_{zz}(\theta)$ using the CNOT and Pauli-Z gate.

$R_x(\theta)$ and $R_z(\theta)$ gates. Particularly, in order to enhance the expressibility and entangling capability of the ansatz, the $R_{zz}(\theta)$ gate can be used as the fundamental two-qubit gates in the circuit,

$$U_{2q} = R_{zz}(\theta) = \exp(i\theta Z_j Z_k), \quad (6)$$

where Z_j and Z_k are the Pauli-Z operators on the j th and k th qubit, respectively. The $R_{zz}(\theta)$ gate can be implemented by using the CNOT and Pauli-Z gates, as illustrated in Fig. 4.

Furthermore, the two-qubit gates need to be arranged in a regular manner to enhance the expressibility and entangling capability of the entire circuit. There are three main configurations of two-qubit gates: nearest-neighbor (NN), circuit-block (CB), and all-to-all (AA) structures, as depicted in Fig. 5. On the one hand, these configurations are proposed to accommodate various quantum hardware platforms with different qubit topologies. Specifically, the NN circuit is the most natural configuration for quantum devices with a linear array of qubits, while AA requires a fully connected architecture of qubits. On the other hand, as discussed in Ref. Sim, Johnson, and Aspuru-Guzik (2019), the three configurations are distinguished from each other in the terms of expressibility, entangling capability, and circuit cost. When having the same number of two-qubit gates (e.g., $D = 4$ for NN, $D = 3$ for CB, and $D = 1$ for AA in Fig. 5), the NN circuit has the poorest expressibility and entangling capability, but the lowest circuit depth. In contrast, AA has the best expressibility and entangling capability, but the highest requirement of circuit depth and connectivity. The CB circuit can provide a good balance, with a much cheaper circuit, while its expressibility and entangling capability are respectively slightly less and equal to AA.

Finally, put all the things together, including the single-qubit gates in Eq. (5), the two-qubit gates in Fig. 4 and the configurations of two-qubit gates in Fig. 5, then the ansatz circuits are obtained. Fig. 6 shows an example of an ansatz circuit with six qubits, which uses the CB configuration of two-qubit gates. It is worth mentioning that, after a pre-experimental study, we determined this circuit as the QRNN's ansatz in the following experiments, as it provides a good balance between circuit complexity and the model's learning performance.

2.3. Partial quantum measurement

The final step of the recurrent block is to output a prediction y_t of the current stage, and maintain an intermediate state h_t that contains information about the history of the sequential data. This is achieved by implementing the partial quantum measurement, namely measuring a portion of qubits as shown in Fig. 2.

Quantum measurement corresponds to a physical observable M , which can be decomposed as $M = \sum_i \lambda_i P_i$ with λ_i being the i th eigenvalue and P_i the projector on the corresponding eigenspace. According to the Born rule, the outcome of the measurement corresponds to one of the eigenvalues λ_i ; that is, the quantum state of the qubits $|\varphi\rangle$ randomly collapse to the corresponding eigenstates $|\lambda_i\rangle$ with a probability $p(\lambda_i) = \langle\varphi|P_i|\varphi\rangle$. Then, the expectation value of the measurement outcome can be formalized as

$$\langle M \rangle = \sum_i \lambda_i p(\lambda_i) = \sum_i \lambda_i \langle\varphi|P_i|\varphi\rangle. \quad (7)$$

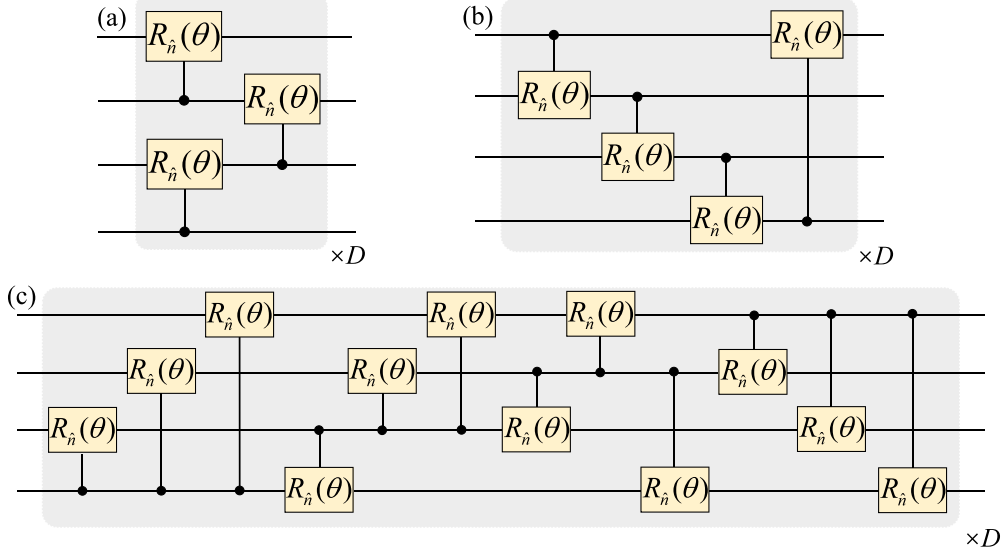


Fig. 5. The circuits for the three configurations of two-qubit gates, (a) nearest-neighbor (NN), (b) circuit-block (CB), and (c) all-to-all (AA).

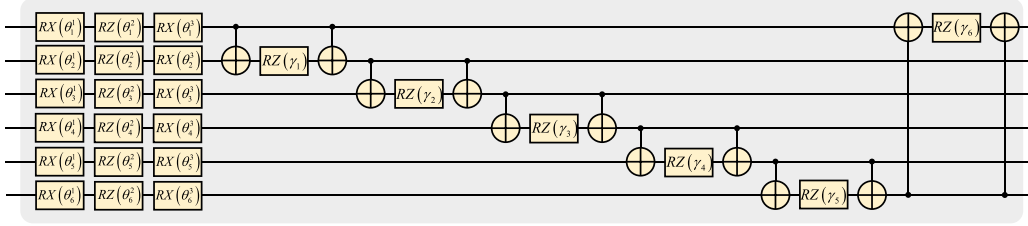


Fig. 6. An example of an ansatz circuit that uses the CB configuration of two-qubit gates.

The most straightforward and commonly used measurement in quantum algorithms is the computational basis measurement, namely the Pauli-Z measurement with the observable σ_Z ,

$$\sigma_Z = (+1)|0\rangle\langle 0| + (-1)|1\rangle\langle 1|. \quad (8)$$

When acting on multiple qubits, Pauli-Z measurement measures whether the individual qubits are in state $|0\rangle$ or $|1\rangle$, from which we can read off the eigenvalues $+1$ or -1 , respectively.

The expectation value $\langle \sigma_Z \rangle$ of a single-qubit is a value in the range $[-1, 1]$. Note that every time we measure a quantum state $|\varphi\rangle$, it will collapse to $|0\rangle$ or $|1\rangle$. Hence, in practice, the expectation is estimated by repeating the operations of creating the state $|\varphi\rangle$ and measuring for S time, where S is also known as the number of shots. The average of the S results is taken as an estimate of the expectation. Therefore, high-precision estimates of the expectation require a larger number of shot, namely rerunning the algorithm many times. It can be proved that the scaling of S is $O(1/\varepsilon^2)$, where ε is the error of the estimation (Schuld & Petruccione, 2021).

In the partial quantum measurement as shown in Fig. 2, only the quantum Reg. D is measured. Specifically, we first implement the Pauli-Z measurement only on the first qubit of Reg. D. The probability of the first qubit collapsing to state $|1\rangle$ is estimated, and after minor post-processing it is taken as the prediction y_t . Then, all the qubits of Reg. D are measured and reinitialized to the state $|0\rangle$ to be ready to embed the next element of the sequential data.

There are two reasons why only the first qubit of Reg. D is measured, rather than all global qubits. Firstly, measuring fewer

qubits can significantly reduce the measurement error when implementing the algorithm on quantum devices. Secondly, global measurement may result in a barren plateau (BP), where the cost function gradients vanish exponentially with the number of qubits (Cerezo, Sone, Volkoff, Cincio and Coles, 2021). By measuring only one qubit, we can reduce the BP impact, and thus improve the trainability of the network.

3. Quantum recurrent neural networks

After obtaining the quantum recurrent block, we can immediately construct the quantum recurrent neural networks by stacking the blocks with certain rules. Below, we first present two architectures of QRNNs, and then discuss the method of optimizing the parameters of QRNNs.

3.1. Two QRNN architectures

The most straightforward way to build QRNNs is by arranging the QRB in sequence, as illustrated in Fig. 7. This architecture is referred to as pQRNN, or plain QRNN. As discussed in Section 2, each QRB follows such process: first, Reg. D embeds one element of the sequential data into the circuit; then, Reg. D and Reg. H are entangled through the ansatz; the first qubit of Reg. D is measured to obtain an intermediate prediction; and finally, Reg. D reinitializes to the state $|0\rangle$ while Reg. H feeds its state directly into the next QRB to transmit the history information of the sequential data.

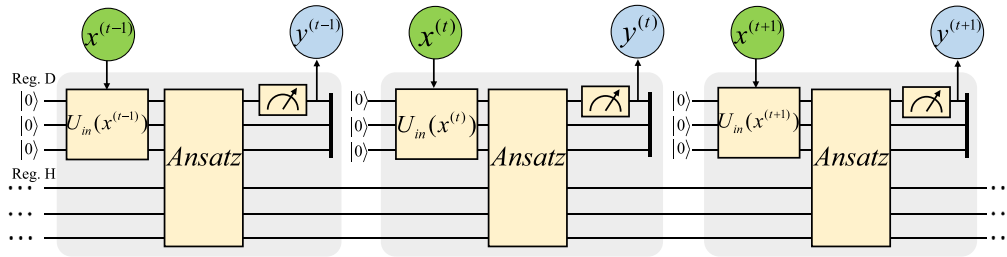


Fig. 7. A straightforward way to build QRNNs using the QRB. This architecture is referred to as pQRNN, or plain QRNN.

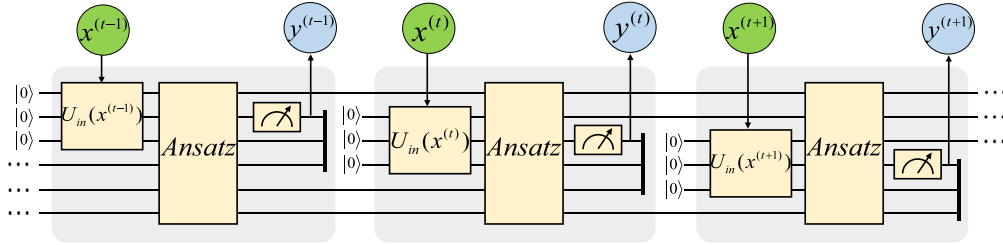


Fig. 8. The second architecture of QRNN, which is built by arranging the QRB in a staggered manner. This model is called sQRNN, or staggered QRNN.

Table 1

Accuracy of the two QRNN models and the classical RNN on the sequential data of meteorological indicators.

Meteorological indicators	Prediction accuracy		
	pQRNN	sQRNN	RNN
Atmospheric pressure	99.93%	99.36%	99.67%
Minimum temperature	96.72%	92.89%	92.65%
Maximum temperature	96.99%	90.35%	92.14%
Relative humidity	98.14%	96.01%	93.35%
Wind speed	90.69%	87.26%	77.44%

Table 2

Prediction accuracy of the pQRNN with 4, 6 and 8 qubits in Reg. D (and Reg. H).

Meteorological indicators	Prediction accuracy		
	4-qubits	6-qubits	8-qubits
Atmospheric pressure	99.94%	99.93%	99.99%
Minimum temperature	94.89%	96.72%	97.07%
Maximum temperature	97.05%	96.99%	98.08%
Relative humidity	98.32%	98.14%	98.30%
Wind speed	87.57%	90.69%	91.12%

In pQRNN, the qubits assigned to Reg. D and Reg. H are fixed. That is, when implementing pQRNN, the qubits of Reg. H need to work at all times. Therefore, pQRNN necessitates quantum devices with long coherent times. However, coherent time is one of the central indicators of NISQ devices, and it is extremely difficult to significantly increase.

In order to address the issues of pQRNN, we propose a new architecture for QRNN. In this model, the QRBs are arranged in a staggered manner, as shown in Fig. 8. This architecture is called sQRNN, or staggered QRNN. In sQRNN, qubits are alternatively assigned to Reg. H, so each qubit has the opportunity to stop working and reinitialize to state $|0\rangle$ after several time steps. By using the strategy of shift work, sQRNN significantly reduces the requirement of coherent time of quantum hardware, making it more accessible on near-term quantum devices.

3.2. Parameters learning

With the two architectures of QRNNs, now we discuss the method of learning optimal parameters of QRNNs. First, predictions (i.e., outcome of quantum measurement) of different discrete time steps are rescaled to associate with the real values or labels. The rescaled predictions are

$$\tilde{y}_t = y_t \cdot (x_{\max} - x_{\min}) + x_{\min}, \quad (9)$$

where $x_{\min}$ and $x_{\max}$ are the minimum and maximum of input states.

Just like classical neural networks, the errors between the predictions and the real values are quantified by the loss functions. Learning parameters is done by minimizing the errors. There are various loss functions to use, which create different landscapes with different properties of plateaus, saddle points, and global minima, etc. The widely used methods include the mean squared error (i.e., L_2 loss) and the cross-entropy loss. In the present work, we use the most straightforward L_2 loss,

$$L_2(\vec{\theta}) = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^T (\tilde{y}_t(x, \vec{\theta}) - y_{true})^2. \quad (10)$$

In this equation, $\vec{\theta}$ represents the parameters to be learned, which are the rotation angles in ansatz circuit; T is the number of time steps; and N is the number of data samples used in the batch training process.

Just like classical neural networks, the parameters can be optimized based on the gradient of the loss function. That is, the parameters are updated towards the direction of steepest descent of the loss function. Gradient-based approaches play a crucial role in deep learning, because there exists the backpropagation (BP) algorithm, which can implement the calculation of the derivatives of the loss function with respect to the parameters through a computationally efficient organization of the chain rule. However, in quantum computing, there exists no similar BP algorithm to evaluate the derivatives. This is because the BP algorithm relies on storing the intermediate state of the network during computation, which is forbidden by the quantum no-cloning theorem.

In quantum deep learning, there are two typical methods to compute the derivatives of the parameters, namely the difference method and the analytical method. In the difference method, the partial derivatives are approximated by the finite difference scheme,

$$\frac{\partial L_2(\vec{\theta})}{\partial \theta_j} = \frac{L_2(\vec{\theta} + \Delta \cdot \vec{e}_j) - L_2(\vec{\theta} - \Delta \cdot \vec{e}_j)}{2\Delta}, \quad (11)$$

where Δ is a tiny hyper-parameter, and $\vec{e}_j$ is a unit vector with the j th element being 1 and the rest being 0. Note that in order to estimate the derivative of each parameter, this method requires to evaluate the loss function twice, that is, implementing the quantum circuit twice.

In the analytical method, analytical gradients can be obtained based on the feature of the quantum gate used in the PQC. Suppose the trainable gates in PQC are of the form $U(\theta_j) = e^{-i\theta_j P_j}$, where P_j is a tensor product of Pauli matrices (in fact, almost all of the PQCs in literature apply this form of quantum gates). Then the derivative of the measurement expectation value (i.e., Eq. (6)) respect to the parameter θ_j can be formalized as (Benedetti et al., 2019)

$$\frac{\partial \langle M \rangle}{\partial \theta_j} = \frac{\langle M \rangle_{\vec{\theta} + \frac{\pi}{2} \cdot \vec{e}_j} - \langle M \rangle_{\vec{\theta} - \frac{\pi}{2} \cdot \vec{e}_j}}{2}, \quad (12)$$

where the subscript $\vec{\theta} \pm \frac{\pi}{2} \cdot \vec{e}_j$ indicate that the parameter θ_j add (minus) $\pi/2$. That is, the derivative is estimated by executing the two circuits with shifted parameter vector. Thus, this formula is also known as the parameter shift rule (Schuld et al., 2020). Furthermore, the derivative of θ_j for the loss function can be obtained by the chain rule,

$$\frac{\partial L_2(\langle M \rangle)}{\partial \theta_j} = \frac{\partial L_2(\langle M \rangle)}{\partial \langle M \rangle} \frac{\partial \langle M \rangle}{\partial \theta_j}. \quad (13)$$

Here we would like to remark that the estimation of the derivatives is performed in a quantum–classical hybrid way. Specifically, the terms $L_2(\vec{\theta} \pm \Delta \cdot \vec{e}_j)$ in Eq. (11) or the terms $\langle M \rangle_{\vec{\theta} \pm \frac{\pi}{2} \cdot \vec{e}_j}$ in Eq. (12) are evaluated by executing the quantum circuit, while the arithmetic with regard to these terms, including Eq. (13), is done in the classical computer. Additionally, as can be seen above, estimating the gradients of the parameters is rather cost in quantum neural networks. Further works are required to clarify the limitation brought by the quantum gradient estimation and design a quantum backpropagation algorithm (Benedetti et al., 2019).

Having the gradients of the parameters, the commonly used optimizers in classical neural networks can be used to update these parameters. Note that this step is performed in classical computers. We test the gradient descent and Adam optimizer. The numerical experiments show that both optimizers can work well; for comparison, Adam can give a 32% improvement in the training speed, while has a slight decrease in accuracy.

4. Experimental results

To evaluate the performance of our QRNN models, we conduct tests on three distinct types of classical sequential data: meteorological indicators, stock prices, and natural language. Both pQRNN and sQRNN architectures are assessed in the numerical experiments. The parameters of QRNN are updated using the difference method for estimating gradients and the gradient descent optimizer. The QRNN algorithms are implemented using the pyQPanda quantum programming framework (PyQPanda, 2023). The datasets used in the experiments can be found in QRNN (2023).

The architecture of the pQRNN and sQRNN used in the experiments is illustrated in Figs. 7 and 8, respectively. The circuits for

data encoding, ansatz and quantum measurement are detailed in Section 2. Both Reg. D and Reg. H utilize three qubits, so the QRNN circuit use six qubits. Each QRB utilizes 24 single-qubit gates and 12 two-qubit gates, so the gate complexity is 36T, where T is the number of QRB. The number of QRB in the QRNN is dependent on the specific dataset being learned.

The classical RNN used as a benchmark in the experiments is developed based on the standard recurrent layer packaged in TensorFlow (TensorflowRNN, 2023). The one recurrent layer has a structure as expressed by Eq. (1), where the activation function f_h uses the tanh function and f_y uses the softmax function. To further enhance the performance of the RNN, both dropout and dense layers are used following the softmax function. The optimized RNN has a total of 49 parameters, which is larger than the 30 parameters of the QRNN. The RNN algorithms are implemented using the TensorFlow framework.

4.1. Meteorological indicators

The meteorological data was obtained from the website of China Meteorological Data Service Center (China Meteorological Data Service Centre, 2023). It includes five indicators: atmospheric pressure, maximum temperature, minimum temperature, relative humidity, and wind speed. The sequence of each indicator contains 500 elements representing the values of the indicator on 500 days, with one data point per day. In the experiments, the 500 elements of each indicator are divided into 300 for training and 200 for test.

The task is to train the QRNN model to predict the value of each indicator on the eighth day using the preceding seven days' values. Therefore, the number of QRB of QRNN is set to seven to embed the seven days' data, and the output of the seventh QRB is taken as the prediction of data on the eighth day. Similarly, the number of recurrent layers of RNN is also seven.

The prediction accuracy of each indicator is evaluated using the following formula,

$$\text{Accuracy} = \left(1 - \sqrt{\frac{1}{N} \sum_{i=1}^N E_i^2} \right) \times 100\%, \quad (14)$$

$$E_i = \frac{\text{actual} - \text{predicted}}{\text{actual}},$$

where E_i is the relative error of each predicted value (i.e., the data of eighth day) and N is the total number of predictions (i.e., the number of samples for test).

The prediction accuracy of the two QRNN models as well as the classical RNN are shown in Table 1. The first remarkable conclusion obtained from the table is that the sQRNN model can achieve a similar accuracy as pQRNN. This is impressive because sQRNN can be implemented efficiently on the NISQ devices, and together with the great performance implies that sQRNN has the potential to find useful applications on near-term quantum devices.

The second remarkable conclusion is that both two QRNN models are capable of predicting the five indicators with higher accuracy than the classical RNN. In particular, for the indicator of wind speed, the performances of QRNN are much better than the classical RNN. The relative variation of wind speed is much acuter than other indicators. Hence, it implies that QRNNs should possess a better capability of predicting the trend of acute variation.

Let us make a further comparison between QRNN and the classical RNN. In QRNN, the number of parameters required to learn are 30, while in RNN it is 49. That is, QRNNs use less parameters but get a higher prediction accuracy. This should result from the fact that PQCs have stronger expressibility than the classical networks (Du, Hsieh, Liu, & Tao, 2020; Du, Tu, Yuan, & Tao,

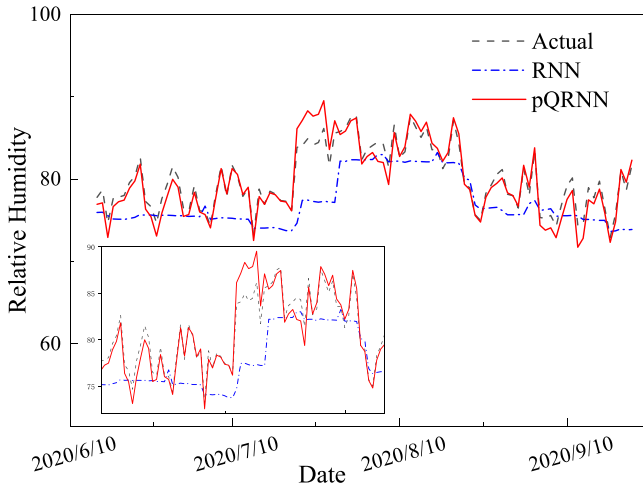


Fig. 9. Variation curves of the relative humidity indicator predicted by pQRNN and RNN. The x-axis represents the date of the values, with one data point per day.

2022). In other words, with the similar number of parameters, the function space achieved by PQC is exponentially larger than that of classical networks. Furthermore, we find that the QRNNs are capable of providing better predictions of the variation details of the temporal data than the classical RNN. Fig. 9 shows the curves of relative humidity indicator predicted by pQRNN and RNN. As can be seen from the figure, Details of the fluctuation presented in the actual sequence data are learned by QRNN, while classical RNN has an effect of smoothing the fluctuation. This, together with the above phenomena that QRNNs perform much better when predicting the wind speed whose variation is much acuter, shows that our QRNNs can capture the changing details existing in the temporal sequence data more efficiently. More predicted curves of other indicators are presented in Appendix A.

In order to verify the flexibility of the circuit structure of QRNN, we test the prediction accuracy of pQRNN with different number of qubits. Specifically, the number of qubits used in Reg. D (also in Reg. H) is set to 4, 6, and 8. The results are shown in Table 2. As can be seen from the table, generally higher prediction accuracy can be obtained when using more qubits in the QRNN circuit. However, there exists the phenomena of accuracy saturation; that is, with the increase of the number of qubits, the improvement of accuracy would become small.

The task presented above involves using the preceding seven days' values to predict the value on the eighth day. Next, we demonstrate that QRNNs are capable of predicting indicator values for multiple consecutive days. In this case, the predicted output of the previous step is used as input to predict the next day's value. Specifically, in the testing phase, we use ten day's values to predict the next five days' value. The results are shown in Table 3. As can be seen from the table, both QRNN models achieve high prediction accuracy, surpassing that of RNN. In particular, QRNN performs significantly better than RNN in predicting wind speed indicator, which has a much sharper variation trend. These results demonstrate QRNN's ability for both consecutive prediction and sharper variation trend prediction.

4.2. Stock price

The second task is to predict the variation of stock's price, including its opening price, maximum price, minimum price, closing price, and volume. The data was obtained from the website

Table 3

Continuous prediction accuracy of the two QRNN models and RNN on meteorological indicators.

Meteorological indicators	Prediction accuracy		
	pQRNN	sQRNN	RNN
Atmospheric pressure	99.83%	98.97%	98.32%
Minimum temperature	91.71%	88.79%	86.49%
Maximum temperature	90.05%	86.45%	86.41%
Relative humidity	90.99%	87.34%	85.76%
Wind speed	83.21%	80.88%	67.99%

Table 4

Prediction accuracy of the two QRNN models and the classical RNN on the sequential data of stock price.

Stock indicators	Prediction accuracy		
	pQRNN	sQRNN	RNN
Opening price	98.23%	97.68%	96.17%
Highest price	98.93%	97.45%	96.99%
Lowest price	98.84%	96.98%	97.04%
Closing price	98.78%	96.81%	96.41%
Volume	90.54%	86.77%	77.73%

of Finance Sina (2023). The sequence data of each stock price component contains 180 elements representing the price of 180 days, with one data point for each weekday. The goal is to use the first seven days' data to predict the price on the eighth day. Therefore, the pQRNN and sQRNN circuits used in this task, as well as the classical RNN, are the same as those used in predicting meteorological indicators. The 180 elements of each sequence are divided into 100 for training and 80 for test. The hyperparameter of learning rate is set to 0.03.

The prediction accuracy of the two QRNN models as well as the classical RNN are shown in Table 4. Almost the same conclusions can be obtained: (1) the sQRNN model can achieve a similar accuracy as pQRNN; (2) both two QRNN models are capable of predicting the five components of stock price with higher accuracy than the classical RNN. Therefore, the advantages of QRNN against classical RNN can be embodied on different kinds of learning data. The predicted curves of five components of stock price are presented in Appendix B.

We also demonstrate the ability of QRNNs to consecutively predict stock prices, similar to meteorological indicators, by using the previous ten day's values to predict the next five days' value. The results are presented in Table 5. As shown, both QRNN models achieve high accuracy in continuous prediction, surpassing that of RNN. In particular, QRNN performs significantly better than RNN in predicting sharper variation trend, i.e. the volume indicator, as observed in meteorological indicators.

In order to further benchmark the efficiency of our QRNNs, we evaluate their performance in predicting the S&P 500 stock market index. This learning task is obtained from Ref. Qiu, Wang, and Zhou (2020) for ease of comparison. The results are presented in Fig. 10. As shown, our QRNNs achieve better performance than the classical methods presented in Ref. Qiu et al. (2020). In particular, QRNNs exhibit faster convergence during the training phase, taking only 100 iterations to achieve convergence, in contrast to the 600 iterations required by classical models.

4.3. Text categorization

The above two tasks are regression problems, while the third task is a classification problem. In order to verify the feasibility

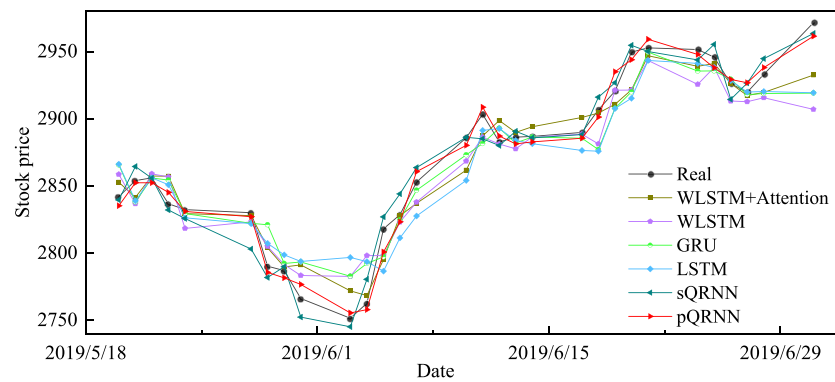


Fig. 10. Variation curves of the S&P 500 opening price predicted by QRNN and classical recurrent models. The x-axis represents the date of the values, with one data point for each weekday.

Table 5

Continuous prediction accuracy of the two QRNN models and RNN on stock price.

Stock indicators	Prediction accuracy		
	pQRNN	sQRNN	RNN
Opening price	96.06%	93.67%	93.73%
Highest price	96.80%	94.72%	92.59%
Lowest price	96.53%	93.98%	92.07%
Closing price	96.34%	94.61%	94.12%
Volume	84.66%	81.13%	79.13%

Table 6

Classification accuracy of the two QRNN models and two state-of-the-art QNN models for sequential learning on the MC task.

Prediction accuracy			
pQRNN	sQRNN	QSANN (Li et al., 2022)	DisCoCat (Lorenz et al., 2023)
100%	100%	100%	79.8%

of QRNNs to perform nature language processing, we use the MC (meaning classification) task to test our QRNNs. In the MC task, there contains 130 sentences and each sentence has 3 or 4 words (Lorenz, Pearson, Meichanetzidis, Kartsaklis, & Coecke, 2023). Half of the sentences are related to food and half to information technology (IT). Hence, MC is a binary classification task that categorizes a sentence as food or IT. There are totally 17 words in MC and part of the vocabulary is in common between the two classes, so the task is not trivial.

Both the number of QRB of QRNN and the number of recurrent layers of RNN is set to four to process the sentences. The 130 sentences are divided into 100 for training and 30 for test. The hyperparameter of learning rate is 0.01.

The classification accuracy of the two QRNN models as well as two state-of-the-art QNN models for natural language learning are shown in Table 6. As can be seen from the table, both two QRNN models can achieve an accuracy of 100%, which is much better than that of the syntactic analysis-based quantum model (Lorenz et al., 2023). On the other hand, the accuracy of QSANN (quantum self-attentive neural networks) proposed in Ref. Li, Zhao, and Wang (2022) also achieves 100%, but the computational cost of QSANN is much heavier than the present QRNNs. In addition, QSANN is a quantum–classical hybrid model, where the queries, keys, and values are implemented by the PQCs and the self-attention coefficients are calculated in the classical resources. QSANN uses 12 quantum parameters to encode each word into the circuit, while in our QRNNs, only one parameter is enough.

5. Conclusion

In the present work, we develop a hardware-efficient way of constructing the quantum recurrent blocks, and by stacking the blocks in a staggered way we obtain the staggered QRNN model that can be implemented efficiently on quantum devices with much lower requirement of the coherent time. The efficiency of the present QRNN models are verified using three different kinds of classical sequential data, i.e., the meteorological indicators, stock price, and text categorization. The numerical experiments show that our QRNN models have a much better performance in prediction (classification) accuracy than the classical RNN and state-of-the-art QNN models for sequential learning, and can capture the acute variation trend existing in the temporal sequence data. In one word, the present QRNNs possess a simple and well-designed structure, and show great performance on both regression and classification problems.

We believe that the present work represents a significant starting point for the study of quantum recurrent neural networks for sequential data. The present QRNN model could serve as a candidate for a canonical QRNN model to explore the potential near-term quantum advantageous applications of quantum deep learning. The interesting future work includes (1) further optimizing the present QRNNs, e.g., by applying different data encoding methods; (2) characterizing the trainability of the QRNNs, including the properties of barren plateau and landscape of cost functions; (3) expanding the present models to create a deep recurrent neural network, similar to the LSTM and GRU networks in deep learning.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The present work is supported by the Natural Science Foundation of Shandong Province of China (ZR2021ZD19) and the National Natural Science Foundation of China (Grant No. 12005212). We are grateful to the support from Big Data Center of Marine Advanced Research Institute of Ocean University of China. We also thank the technical team from the Origin Quantum Computing Technology Co., Ltd, in Hefei for their professional services.

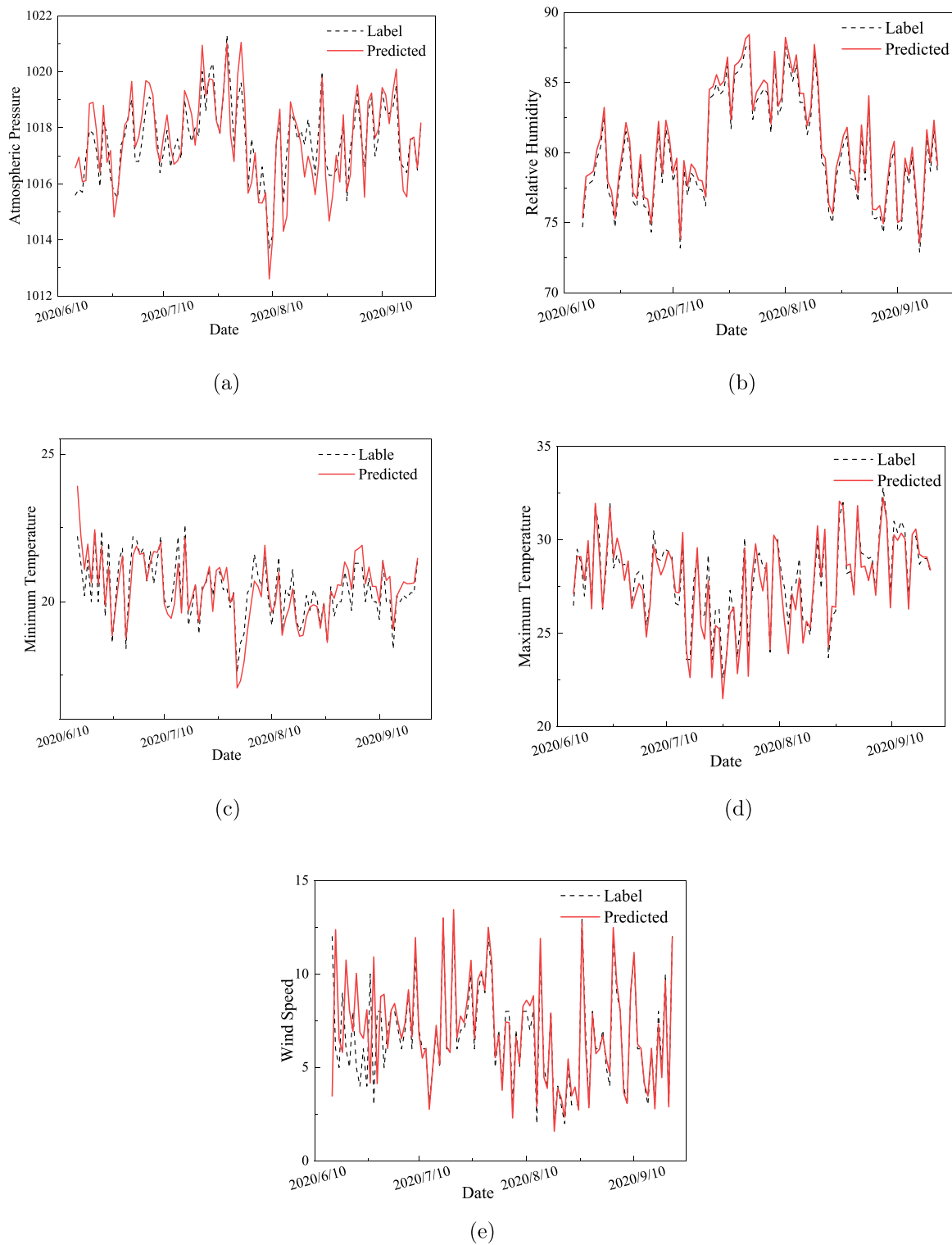


Fig. A.11. Variation curves of the meteorological indicators predicted by pQRNN: (a) atmospheric pressure, (b) relative humidity, (c) minimum temperature, (d) maximum temperature, and (e) wind speed. The x-axis represents the date of the values, with one data point per day.

Appendix A. Curves of the meteorological indicators predicted by pQRNN and sQRNN

A.2. Variation curves of the meteorological indicators predicted by sQRNN

A.1. Variation curves of the meteorological indicators predicted by pQRNN

See Fig. A.12.

See Fig. A.11.

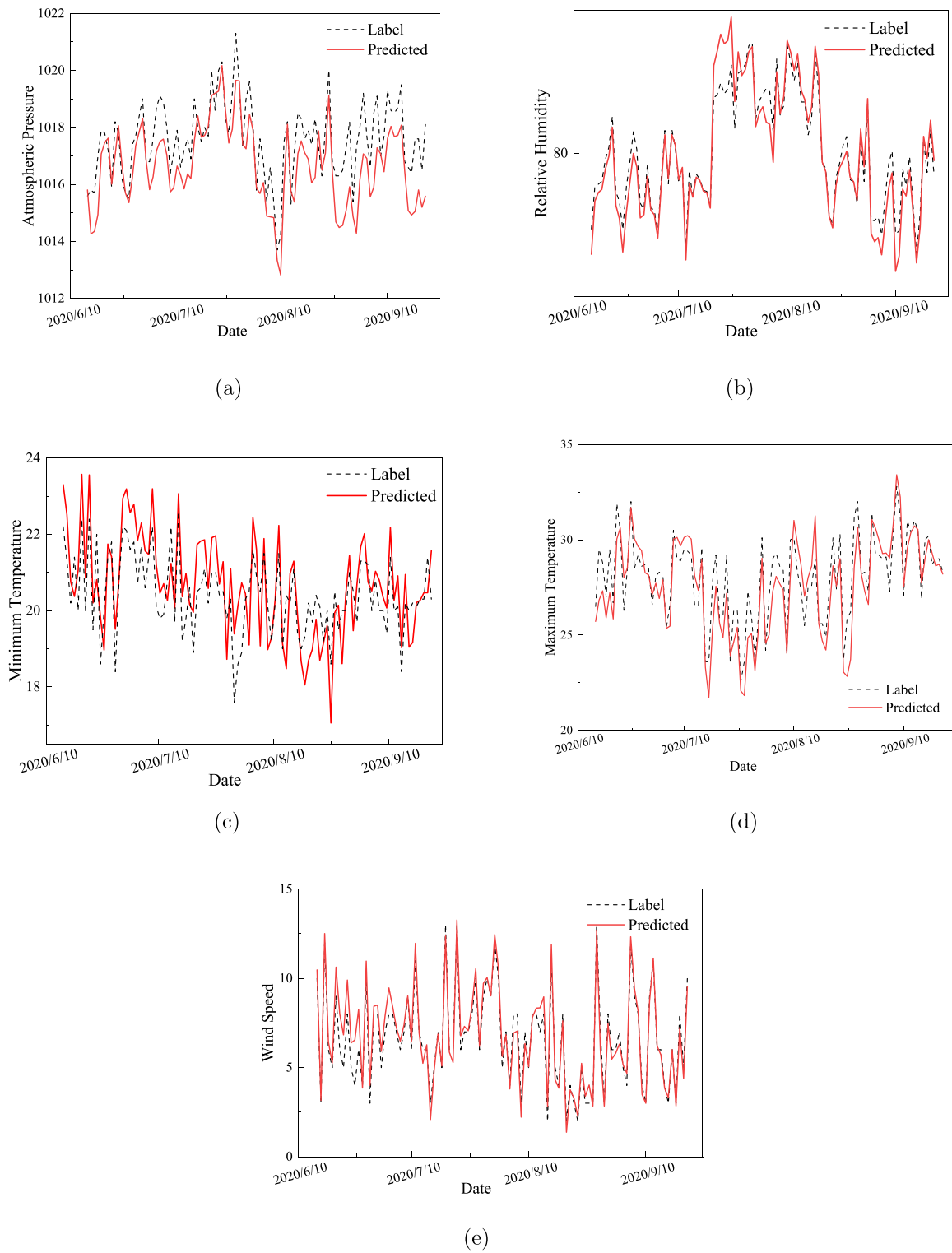


Fig. A.12. Variation curves of the meteorological indicators predicted by sQRNN: (a) atmospheric pressure, (b) relative humidity, (c) minimum temperature, (d) maximum temperature, and (e) wind speed. The x-axis represents the date of the values, with one data point per day.

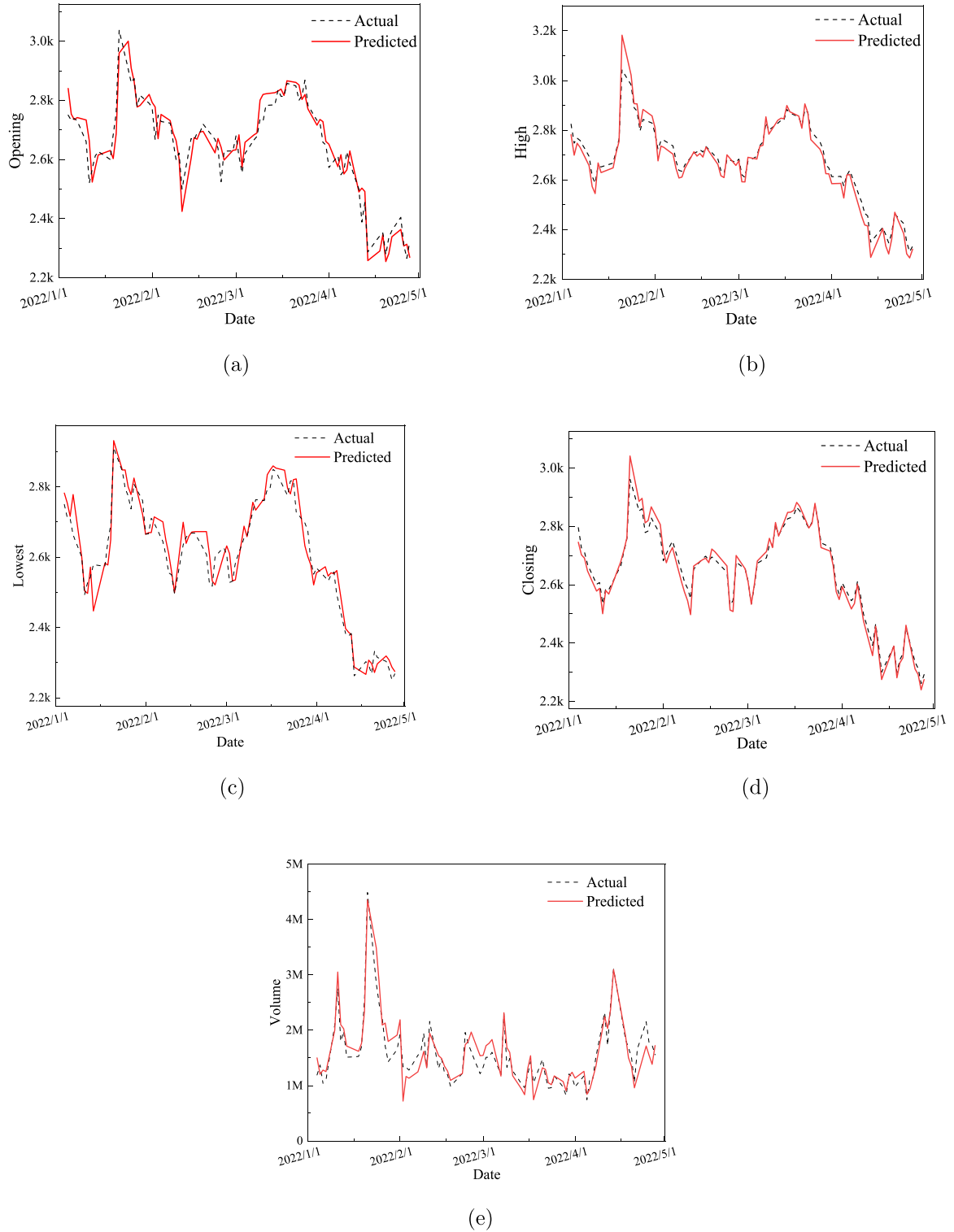


Fig. B.13. Variation curves of the stock price predicted by pQRNN: (a) the opening price, (b) the highest price, (c) the lowest price, (d) the closing price, and (e) the volume. The x-axis represents the date of the values, with one data point for each weekday.

Appendix B. Curves of the stock price predicted by pQRNN and sQRNN

B.2. Variation curves of the stock price predicted by sQRNN

B.1. Variation curves of the stock price predicted by pQRNN

See Fig. B.13.

See Fig. B.14.

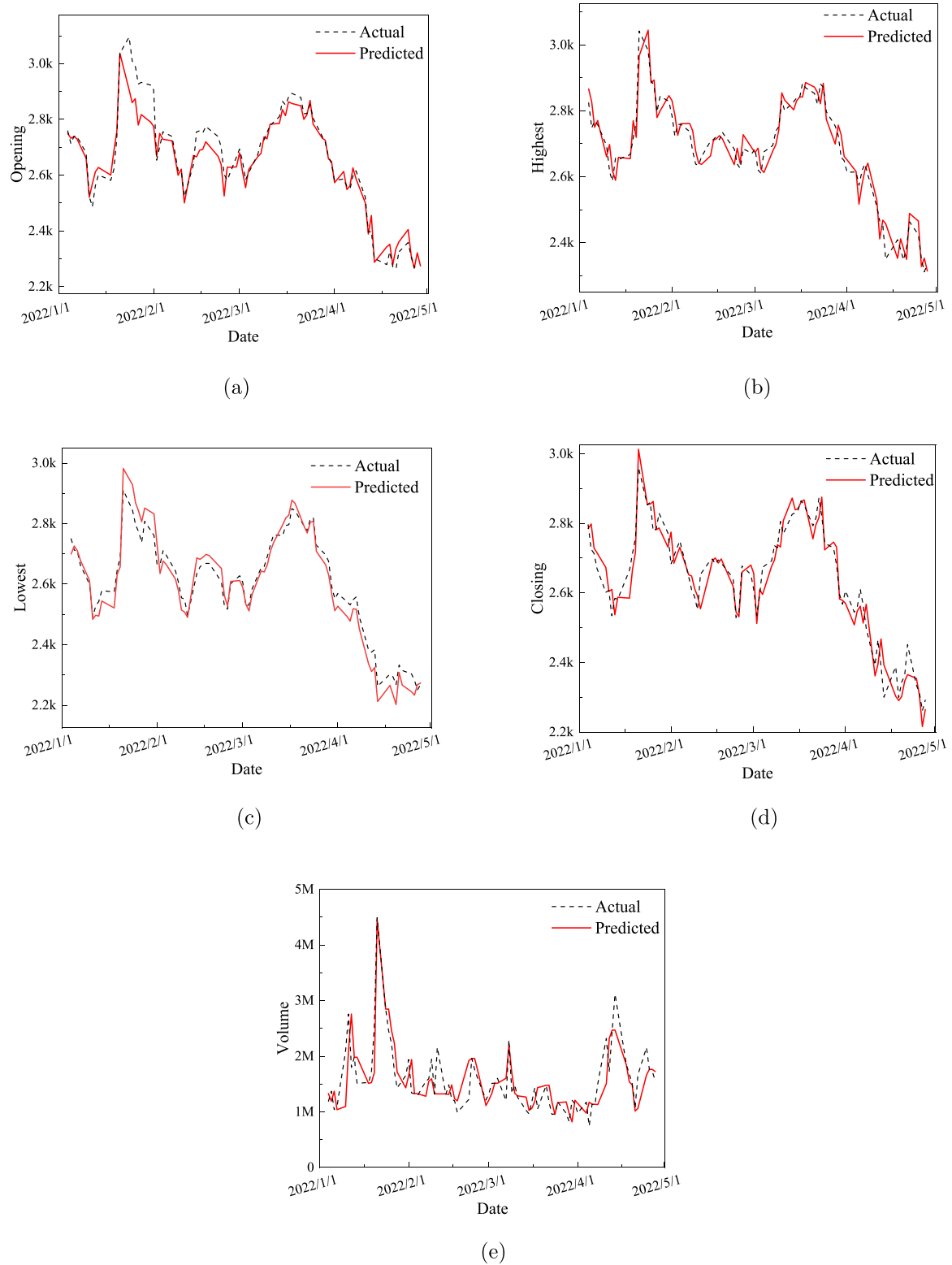


Fig. B.14. Variation curves of the stock price predicted by sQRNN: (a) the opening price, (b) the highest price, (c) the lowest price, (d) the closing price, and (e) the volume. The x-axis represents the date of the values, with one data point for each weekday.

References

- Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J. C., Barends, R., et al. (2019). Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779), 505–510. <http://dx.doi.org/10.1038/s41586-019-1666-5>.
- Bausch, J. (2020). Recurrent quantum neural networks. In *Advances in neural information processing systems*, Vol. 33 (pp. 1368–1379). Curran Associates, Inc..
- Benedetti, M., Lloyd, E., Sack, S., & Fiorentini, M. (2019). Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4), Article 043001. <http://dx.doi.org/10.1088/2058-9565/ab4eb5>, arXiv:1906.07682.
- Bharti, K., Cervera-Lierta, A., Kyaw, T. H., Haug, T., Alperin-Lea, S., Anand, A., et al. (2022). Noisy intermediate-scale quantum algorithms. *Reviews of Modern Physics*, 94(1), Article 015004. <http://dx.doi.org/10.1103/RevModPhys.94.015004>.
- Cai, H., Ye, Q., & Deng, D.-L. (2022). Sample complexity of learning parametric quantum circuits. *Quantum Science and Technology*, 7(2), Article 025014. <http://dx.doi.org/10.1088/2058-9565/ac4f30>.
- Cao, Y., Guerreschi, G. G., & Aspuru-Guzik, A. (2017). Quantum neuron: an elementary building block for machine learning on quantum computers. arXiv:1711.11240.
- Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., et al. (2021). Variational quantum algorithms. *Nature Reviews Physics*, 3(9), 625–644. <http://dx.doi.org/10.1038/s42254-021-00348-9>, arXiv:2012.09265.
- Cerezo, M., Sone, A., Volkoff, T., Cincio, L., & Coles, P. J. (2021). Cost function dependent barren plateaus in shallow parametrized quantum circuits. *Nature Communications*, 12(1), 1791. <http://dx.doi.org/10.1038/s41467-021-21728-w>.
- Chen, S. Y.-C., Yoo, S., & Fang, Y.-L. L. (2022). Quantum long short-term memory. In *ICASSP 2022 - 2022 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 8622–8626). Singapore, Singapore: IEEE. <http://dx.doi.org/10.1109/ICASSP43922.2022.9747369>.
- China Meteorological Data Service Centre (2023). <https://data.cma.cn/>.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv:1406.1078.
- Cong, I., Choi, S., & Lukin, M. D. (2019). Quantum convolutional neural networks. *Nature Physics*, 15(12), 1273–1278. <http://dx.doi.org/10.1038/s41567-019-0648-8>, arXiv:1810.03787.
- Di Sipio, R., Huang, J.-H., Chen, S. Y.-C., Mangini, S., & Worring, M. (2022). The dawn of quantum natural language processing. In *ICASSP 2022 - 2022 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 8612–8616). <http://dx.doi.org/10.1109/ICASSP43922.2022.9747675>.
- Du, Y., Hsieh, M.-H., Liu, T., & Tao, D. (2020). Expressive power of parametrized quantum circuits. *Physical Review Research*, 2(3), Article 033125. <http://dx.doi.org/10.1103/PhysRevResearch.2.033125>.
- Du, Y., Tu, Z., Yuan, X., & Tao, D. (2022). Efficient measure for the expressivity of variational quantum algorithms. *Physical Review Letters*, 128(8), Article 080506. <http://dx.doi.org/10.1103/PhysRevLett.128.080506>.
- Farhi, E., Goldstone, J., & Gutmann, S. (2014). A quantum approximate optimization algorithm. arXiv:1411.4028.
- Farhi, E., & Neven, H. (2018). Classification with quantum neural networks on near term processors. arXiv:1802.06002.
- Finance.sina. (2023). <https://vip.stock.finance.sina.com.cn/mkt/>.
- Gil Vidal, F. J., & Theis, D. O. (2020). Input redundancy for parameterized quantum circuits. *Frontiers in Physics*, 8, 297. <http://dx.doi.org/10.3389/fphy.2020.00297>.
- Goldberg, Y. (2016). A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research*, 57, 345–420. <http://dx.doi.org/10.1613/jair.4992>.
- Graves, A. (2014). Generating sequences with recurrent neural networks. arXiv:1308.0850.
- Gu, J., Wang, Z., Kuen, J., Ma, L., Shahroudy, A., Shuai, B., et al. (2018). Recent advances in convolutional neural networks. *Pattern Recognition*, 77, 354–377. <http://dx.doi.org/10.1016/j.patcog.2017.10.013>.
- Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., et al. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747), 209–212. <http://dx.doi.org/10.1038/s41586-019-0980-2>.
- Herrmann, J., Llima, S. M., Remm, A., Zapletal, P., McMahon, N. A., Scarato, C., et al. (2022). Realizing quantum convolutional neural networks on a superconducting quantum processor to recognize quantum phases. *Nature Communications*, 13(1), 4144. <http://dx.doi.org/10.1038/s41467-022-31679-5>.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <http://dx.doi.org/10.1162/neco.1997.9.8.1735>.
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. [http://dx.doi.org/10.1016/0893-6080\(89\)90020-8](http://dx.doi.org/10.1016/0893-6080(89)90020-8).
- Hur, T., Kim, L., & Park, D. K. (2022). Quantum convolutional neural network for classical data classification. *Quantum Machine Intelligence*, 4(1), 3. <http://dx.doi.org/10.1007/s42484-021-00061-x>.
- Jordan, M. I., & Mitchell, T. M. (2015). Machine learning: trends, perspectives, and prospects. *Science*, 349(6245), 255–260. <http://dx.doi.org/10.1126/science.aaa8415>.
- Kandala, A., Mezzacapo, A., Temme, K., Takita, M., Brink, M., Chow, J. M., et al. (2017). Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature*, 549(7671), 242–246. <http://dx.doi.org/10.1038/nature23879>.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <http://dx.doi.org/10.1038/nature14539>.
- Li, G., Zhao, X., & Wang, X. (2022). Quantum self-attention neural networks for text classification. arXiv:2205.05625.
- Lorenz, R., Pearson, A., Meichanetzidis, K., Kartsaklis, D., & Coecke, B. (2023). QNLP in practice: running compositional models of meaning on a quantum computer. *Journal of Artificial Intelligence Research*, 76, 1305–1342. <http://dx.doi.org/10.1613/jair.114329>.
- Madsen, L. S., Laudendach, F., Askarani, M. F., Rortais, F., Vincent, T., Bulmer, J. F. F., et al. (2022). Quantum computational advantage with a programmable photonic processor. *Nature*, 606(7912), 75–81. <http://dx.doi.org/10.1038/s41586-022-04725-x>.
- Mitarai, K., Negoro, M., Kitagawa, M., & Fujii, K. (2018). Quantum circuit learning. *Physical Review A*, 98(3), Article 032309. <http://dx.doi.org/10.1103/PhysRevA.98.032309>.
- Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge ; New York: Cambridge University Press.
- Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., et al. (2014). A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5(1), 4213. <http://dx.doi.org/10.1038/ncomms5213>.
- Pesah, A., Cerezo, M., Wang, S., Volkoff, T., Sornborger, A. T., & Coles, P. J. (2021). Absence of barren plateaus in quantum convolutional neural networks. *Physical Review X*, 11(4), Article 041011. <http://dx.doi.org/10.1103/PhysRevX.11.041011>.
- Peters, E., Caldeira, J., Ho, A., Leichenauer, S., Mohseni, M., Neven, H., et al. (2021). Machine learning of high dimensional data on a noisy quantum processor. *Npj Quantum Information*, 7(1), 161. <http://dx.doi.org/10.1038/s41534-021-00498-9>.
- Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, 2, 79. <http://dx.doi.org/10.22331/q-2018-08-06-79>, arXiv:1801.00862.
- Pyqanda. (2023). <https://pyqanda-tutorial.readthedocs.io/zh/latest/>.
- Qiu, J., Wang, B., & Zhou, C. (2020). Forecasting stock prices with long-short term memory neural network based on attention mechanism. *PLOS ONE*, 15(1), Article e0227222. <http://dx.doi.org/10.1371/journal.pone.0227222>.
- QRNN-datasets. (2023). <https://github.com/Lynn11312/QRNN-datasets>.
- Schuld, M. (2021). Supervised quantum machine learning models are kernel methods. arXiv:2101.11020.
- Schuld, M., Bocharov, A., Svore, K. M., & Wiebe, N. (2020). Circuit-centric quantum classifiers. *Physical Review A*, 101(3), Article 032308. <http://dx.doi.org/10.1103/PhysRevA.101.032308>.
- Schuld, M., & Petruccione, F. (2021). *Quantum Science and Technology, Machine Learning with Quantum Computers*. Cham: Springer International Publishing. <http://dx.doi.org/10.1007/978-3-030-83098-4>.
- Schuld, M., Sweke, R., & Meyer, J. J. (2021). Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3), Article 032430. <http://dx.doi.org/10.1103/PhysRevA.103.032430>.
- Sim, S., Johnson, P. D., & Aspuru-Guzik, A. (2019). Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms. *Advanced Quantum Technologies*, 2(12), Article 1900070. <http://dx.doi.org/10.1002/qute.201900070>.
- Takaki, Y., Mitarai, K., Negoro, M., Fujii, K., & Kitagawa, M. (2021). Learning temporal data with a variational quantum recurrent neural network. *Physical Review A*, 103(5), Article 052414. <http://dx.doi.org/10.1103/PhysRevA.103.052414>.
- TensorflowRNN (2023). TensorflowRNN. https://tensorflow.google.cn/api_docs/python/tf/keras/layers.
- Voulodimos, A., Doulamis, N., Doulamis, A., & Protopapadakis, E. (2018). Deep learning for computer vision: a brief review. *Computational Intelligence and Neuroscience*, 2018, 1–13. <http://dx.doi.org/10.1155/2018/7068349>.
- Zhong, H.-S., Wang, H., Deng, Y.-H., Chen, M.-C., Peng, L.-C., Luo, Y.-H., et al. (2020). Quantum computational advantage using photons. *Science*, 370(6523), 1460–1463. <http://dx.doi.org/10.1126/science.abe8770>.